SUPPLEMENTARY MATERIALS — INDEX

Thesis: Audio-Based Bearing Fault Detection on Resource-Constrained Microcontrollers: An Ablation-Driven Study

Author: Ramin Nassiri

Program: M.Sc. Electrical and Microsystems Engineering (MEM), OTH Regensburg

Supervisors: Prof. Dr.-Ing. Armin Sehr; Prof. Dr.-Ing. Johannes Reschke

Purpose:

This document lists and explains the supplementary files (code, models, figures, results, firmware notes) submitted with the thesis and gives brief reproduction guidance.

Contents

1. Dataset — University of Ottawa (UORED-VAFCLS) 2
2. Data Segmentation 2
3. Feature Extraction and Preprocessing 2
    Core Feature Pipeline 3
    Envelope / Preprocessing-Based Features 3
    Spectral Feature Experiments 3
    Preprocessing Scripts 3
4. Model Training and Ablation Experiments 4
    A-Series Experimental Variants 4
    Segment Aggregation Strategies 4
    Training and Calibration Utilities 4
    B-Series Evaluation 4
5. Model Export and Deployment 5
    Model Export Scripts 5
    Exported Models 5
6. Experimental Results and Evaluation Outputs 5
    Result Plots (Confusion Matrices) 6
    Result Tables 6
7. Complete Confusion Matrix Set 6

1. Dataset — University of Ottawa (UORED-VAFCLS)

Source: Mendeley Data (Version 5) — https://data.mendeley.com/datasets/y2px5tg92h/5
The original files were not WAV. I converted them to **16-bit PCM WAV** for consistent processing. The converted WAV files used in this study and the corresponding Python converter script are included.

Included paths in this ZIP:

- `01_uottawa_wavs/wav/` — converted WAVs used in the study
- `01_uottawa_wavs/converter/` — Python converter script

2. Data Segmentation

I segmented the audio into fixed 2.0 s windows with 0.5 s hop (75% overlap) for training and evaluation. Labels were preserved; no filtering was applied during segmentation. The process also creates train/val/test CSV manifests for reproducibility.

Included .py scripts:

- ingest_uottawa_raw_split.py — loads raw files and prepares initial lists
- prep_segments.py — makes fixed-length segments
- make_splits.py — creates train/val/test splits
- resegment_make_B1_B2.py — alternative B1/B2 re-segmentation

3. Feature Extraction and Preprocessing

This section contains the scripts used to implement the feature extraction pipeline employed throughout the ablation study. Each segmented 2.0 s audio window is processed using optional preprocessing stages, including high-pass filtering, pre-emphasis, notch filtering (50/60 Hz), band-pass filtering, and spectral whitening. Envelope-based representations are computed using the Hilbert transform, followed by logarithmic compression where applicable. Feature representations include time-domain descriptors, frequency-domain descriptors, envelope-log variants at different sampling rates (8 kHz and 16 kHz), and dimensionality reduction using PCA (2, 4, and 8 components). Additional experimental feature sets include log-Mel spectrograms and MFCC-based representations. Feature normalization and numerical stabilization (e.g., NaN handling) are applied prior to model training. No label modifications were performed during this stage.

Core Feature Pipeline

extract_features.py — Main feature extraction pipeline. Computes time and frequency features from each segmented audio file.

feat_a8.py — Implements the A8 feature configuration used in the final pipeline.

fix_feature_nans.py — Handles NaN or invalid feature values to ensure numerical stability before modeling.

make_baseline_time.py — Generates baseline time-domain features for comparison with enhanced pipelines.

Envelope / Preprocessing-Based Features

a10_hilbert_envelope.py — Computes the Hilbert envelope of the signal for amplitude-based feature extraction.

a11_hilbert_envelope_log.py — Applies logarithmic compression to the Hilbert envelope.

a12_env_log_sr8k.py — Envelope + log compression at 8 kHz sampling rate.

a13_env_log_sr8k_pca8.py — Envelope-log features with PCA reduction to 8 components.

Spectral Feature Experiments

a36_logmel20.py — Extracts 20-band log-Mel spectrogram features.

a37_logmel40.py — Extracts 40-band log-Mel spectrogram features.

a38_mfcc20.py — Computes 20 MFCC coefficients per segment.

a39_mfcc20_deltas.py — MFCC features with delta (temporal derivative) coefficients.

Preprocessing Scripts

a1_hpf.py — High-pass filtering
a2_preemph.py — Pre-emphasis filtering
a3_hpf_preemph.py — Combined HPF + pre-emphasis
a4_notch50.py — 50 Hz notch filtering
a6_notch60.py — 60 Hz notch filtering
a8_bandpass_1_5k.py — Band-pass filtering (1–5 kHz)
a9_spectral_whitening.py — Spectral whitening

4. Model Training and Ablation Experiments

This section contains the scripts used to train and evaluate the different model variants investigated in the ablation study (A0–A41 and related experiments). The experiments compare preprocessing strategies, feature configurations, dimensionality reduction, normalization methods, and classifier types (GMM, MLP, Random Forest, Extra Trees). Each script corresponds to a defined experimental configuration used to measure classification performance.

A-Series Experimental Variants

These scripts implement the systematic ablation study (A0–A43), exploring preprocessing, sampling rate, PCA dimensionality, normalization strategy, and classifier choice.

- **A0_baseline.py** — Baseline configuration used as reference.
- **a14–a34** — Envelope-log feature variants with different sampling rates (8/16 kHz), PCA dimensions (2/4/8), and normalization methods (min-max, z-score, robust).
- **a42_rf_on_A16_envlog8k.py** — Random Forest evaluation on A16 features.
- **a43_extratrees_on_A16_envlog8k.py** — Extra Trees evaluation on A16 features.

Segment Aggregation Strategies

These scripts convert segment-level predictions into file-level decisions.

- **a35a_aggregate_A8_GMM_per_file.py**
- **a35b_aggregate_A16_MLP_per_file.py**
- **a35c_aggregate_A31_MLP_per_file.py**
- **a40_aggregate_majority.py**
- **a41_aggregate_median.py**

Training and Calibration Utilities

- **train_models.py** — Main training pipeline
- **tune_models.py** — Hyperparameter tuning
- **train_calibrate.py** — Threshold calibration
- **model_rt_gmm.py** — Real-time oriented GMM training
- **time_gmm.py** — Runtime measurement
- **run_variant_ov75.py** — 75% overlap experiment execution

B-Series Evaluation

- **b1_eval_A16_A8.py** — Comparative evaluation (B1 protocol)
- **b2_eval_A16_A8.py** — Extended comparison (B2 protocol)

5. Model Export and Deployment

This section contains the scripts used to export trained models into lightweight formats suitable for embedded deployment. The exports include GMM models in JSON format and MLP / tree-based models converted to compact header or minimal representations. These files were used for deployment and on-device evaluation.

Model Export Scripts

- **a44_export_only_et_header.py** — Exports the trained Extra Trees model into a lightweight header representation suitable for embedded deployment.
- **a45_train_export_rf_small.py** — Trains a compact Random Forest model and exports it in a reduced format for resource-constrained environments.
- **a46_export_mlp_MIN.py** — Produces a minimal export of the trained MLP model with reduced parameter storage.
- **a46_export_mlp_to_header.py** — Converts the trained MLP model into a C-style header format for integration into embedded systems.
- **a47_export_gmm_json.py** — Exports Gaussian Mixture Model parameters into a portable JSON format for deployment.

Exported Models

- **gmm_A8.joblib** — Serialized Gaussian Mixture Model corresponding to the **A8 pipeline**, selected for embedded deployment due to its computational efficiency.
- **mlp_A16.joblib** — Serialized Multilayer Perceptron model corresponding to the **A16 pipeline**, which achieved the best classification performance in the offline experiments.

6. Experimental Results and Evaluation Outputs

This section contains the result artifacts generated during the ablation study and final model evaluation. These include confusion matrix visualizations for different experimental configurations, summary result tables, and evaluation outputs used to compare model performance across the A-series experiments. Only representative plots and final evaluation tables relevant to the conclusions of the thesis are included.

Result Plots (Confusion Matrices)

- **cm_A8_GMM.png** — Confusion matrix for the A8 GMM pipeline, representing the best-performing lightweight model used for embedded deployment.
- **cm_A16_MLP.png** — Confusion matrix for the A16 MLP pipeline, representing the highest-performing model in the offline experiments.
- **cm_A42_RF.png** — Confusion matrix for the Random Forest model evaluated on the A16 feature configuration.
- **cm_A43_ET.png** — Confusion matrix for the Extra Trees model evaluated on the A16 feature configuration.

Result Tables

- **variants.csv** — Summary table containing the results of all ablation experiments (A-series), including performance metrics such as accuracy and F1-score for each configuration.
- **best_of_table.csv** — Consolidated table highlighting the top-performing configurations across all experiments.
- **ablation_summary.csv** — Simplified overview of the ablation experiments (A-series), presenting key pipeline parameters and corresponding performance metrics (F1-score) in a human-readable format for quick comparison.

7. Complete Confusion Matrix Set

All confusion matrices generated during the ablation study (A0–A43) are included in this section. These matrices cover both GMM and MLP models and provide detailed class-wise performance evaluation for each experimental configuration. This set corresponds to the full appendix of the thesis and ensures complete transparency and reproducibility of the experimental results.